

Co-Farmer: Project Plan

Sep 25, 2026 · Reboot the Earth hackathon · Doha · **Plan v1**

Co-Farmer is a sensing and control kit for small farms, looked after by a team of AI specialists built for each farm. Sensors in every field report to a master node, which logs the data and switches pumps and fans to keep conditions inside safe ranges. A few times a day, an AI agent network reads the farm's data and the weather forecast, sets new ranges, and writes plain-language advice for the farmer. Before any of that, an AI assistant talks with the farmer in their own words and designs the hardware and the agent network for that farm.

Every part of this plan says what we build for the hackathon and what the finished product adds:

Tag	Meaning
MVP	What we build and demo at the hackathon
Final	What the product adds after the hackathon

Section 4 follows one farm from the first conversation to a running system. Section 6 is a full worked example of the agent network.

1. Overview

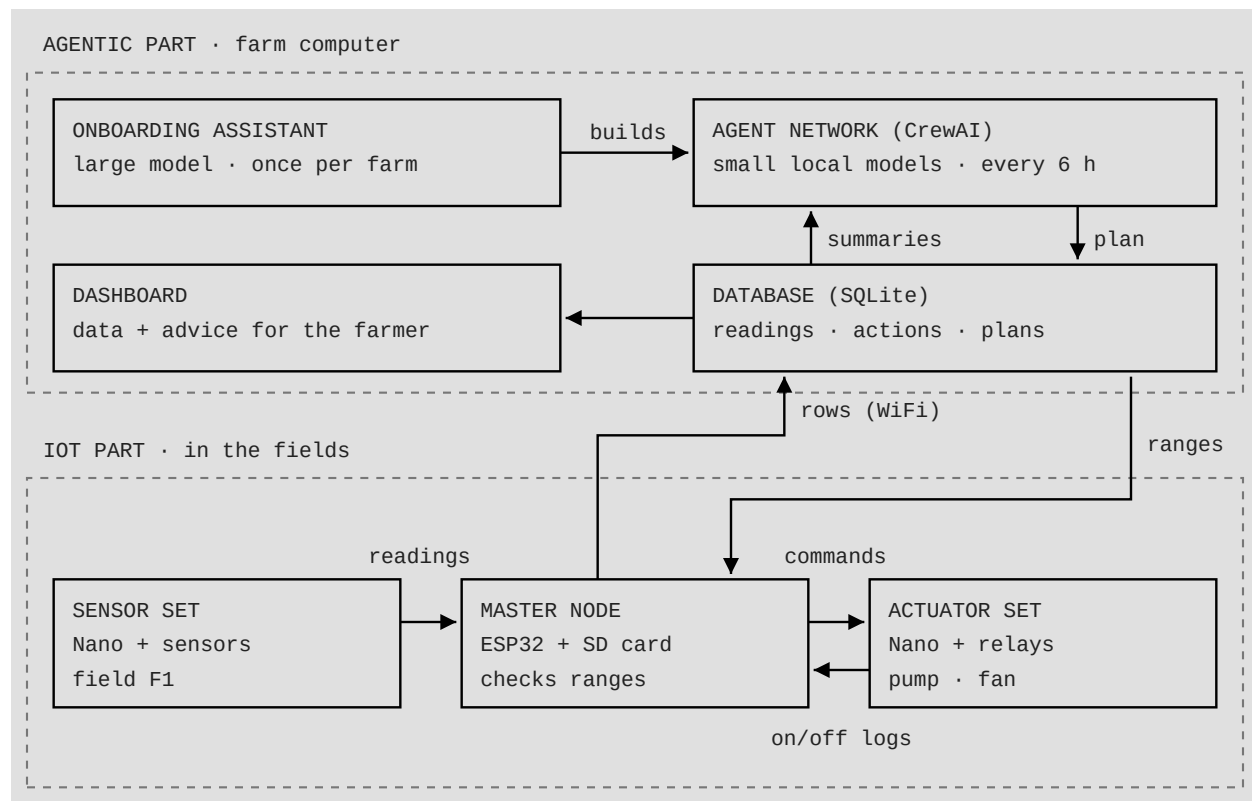


Figure 1. The two parts. The IoT part runs the fields on its own; the agentic part sets the ranges it enforces.

1.1 The IoT part

Three kinds of device work in the fields:

- **Sensor set** (Arduino Nano + sensors, per field): reads its sensors about every 10 seconds and radios the readings to the master.
- **Master node** (ESP32 + SD card, one per farm): stamps each reading with the time, writes it to the SD card and sends it to the farm computer. It checks every reading against the current ranges and, when one is out, commands an actuator set.
- **Actuator set** (Arduino Nano + relays, per field): switches pumps and fans, and reports back each time it turns something on and each time it turns it off.

The IoT part keeps the farm safe on its own. If the farm computer or the internet goes down, the master keeps enforcing the last ranges it received.

1.2 The agentic part

Two AI stages, both reached from the farm computer:

- **Onboarding assistant** (once per farm, a large model): talks with the farmer in natural language, looks up the location and climate, and designs two things: the hardware for each field, and the farm's agent network.
- **Farm agent network** (every few hours, small open models): departments of specialist agents, built with CrewAI, read summaries of the farm's data and the forecast. A Farm Director agent turns their reports into one plan: new ranges for the master, and advice for the farmer.

The agents never switch hardware. They set the ranges; the master enforces them.

1.3 Where the two parts meet

Two small formats connect them, so the hardware and AI work can be built in parallel:

- **Log rows**, IoT → database: every reading and every actuator switch (§2.4).
- **Ranges**, agents → master: the allowed band for each field and variable (§2.5).

1.4 MVP and final product at a glance

Area	MVP (hackathon)	Final product
Farm	One demo field on a table	Many fields per farm, many farms
Devices	One sensor set, one actuator set (pump and fan), one master	Sensor and actuator sets in every field
Data	CSV on the SD card, plus a database on the laptop	Same on the farm, synced to a cloud server
Onboarding	Chat in English with a large model; the team checks the outputs	Arabic and English, text or voice, on a phone

Area	MVP (hackathon)	Final product
Agents	Small open models on the laptop, run by a button and every 6 hours	Specialists fine-tuned on each farm's own records
Dashboard	Streamlit page on the laptop	Web and mobile app, WhatsApp or SMS alerts
Power	USB power banks	Solar, sized in the team's power plan

2. The IoT part in depth

2.1 The devices

Device	Hardware	Job
Sensor set	Arduino Nano Every, nRF24L01 radio. Sensors: air temperature and humidity (DHT11), soil moisture, light, water level, air quality (MQ-135)	Reads its sensors about every 10 s and radios each reading to the master
Actuator set	Arduino Nano, nRF24L01 radio, relay board, pump, fan	Carries out the master's commands; logs "activated" when it switches on and "deactivated" when it switches off
Master node	ESP32, nRF24L01 radio, SD card module	Stamps the time, logs every row to the SD card, sends rows to the farm computer, checks ranges, commands actuator sets, fetches new ranges
Farm computer	Laptop (MVP); a small PC on the farm (final)	Receives rows into the database, runs the dashboard and the agent network. In the MVP, one laptop does all of this

2.2 Naming every device

Every row carries three IDs, so any reading or action can be traced to one place:

ID	Means	Examples
field_id	Which field (farmland) inside the farm	F1, F2
module_id	Which sensor set or actuator set in that field	S1 (sensor set), A1 (actuator set)
device_id	Which sensor or relay on that module	dht11, soil, light, level, mq135; relay1 (pump), relay2 (fan)

The IDs are set in each module's firmware at installation and written into the installation record (§4, step 4).

2.3 What happens every 10 seconds

1. The sensor set reads its sensors and radios one small packet per reading to the master.
2. The master stamps the time, appends a row to the CSV on the SD card, and sends the same row over WiFi to the farm computer, which saves it in the database. If the farm computer can't be reached, rows wait on the SD card and are sent when it's back.
3. The master compares the reading with the current range for that field and variable.
4. If the reading is outside its range, the master commands that field's actuator set, e.g. "relay1 on for 120 s".
5. The actuator set switches the relay and radios back an **activated** log. When it switches off, it sends a **deactivated** log. The master stamps and stores both, like any reading.
6. The dashboard shows the new rows as they arrive.

Why both an SD card and a database: the SD card is the black box that never depends on WiFi; the database is what the dashboard and the agents read. The ESP32 sends rows to a small web endpoint on the farm computer rather than talking to the database directly, which keeps database passwords off the device.

2.4 The log format

One row per reading or per actuator switch. The same columns go to the SD card and the database:

```
field_id,module_id,device_id,type,kind,value,unit,timestamp
F1,S1,dht11,reading,temp_air,30.4,C,2026-09-25T14:00:10+03:00
F1,S1,soil,reading,soil_moisture,29.5,%,2026-09-25T14:00:10+03:00
F1,A1,relay1,action,pump,activated,,2026-09-25T14:00:11+03:00
F1,A1,relay1,action,pump,deactivated,,2026-09-25T14:02:11+03:00
```

- **type** is reading or action; **kind** is what was measured or switched; **value** is the number, or activated / deactivated.
- This holds the same information as the reading/action dictionary the team sketched, split into columns so the CSV stays valid and easy to filter.
- **timestamp** is when the master received the row, in local time with its offset.

2.5 The ranges the master enforces

The agents decide the numbers; the master's firmware decides what each one means. MVP rules:

Reading	Below the minimum	Above the maximum
soil_moisture	Pump on for the set number of seconds	Alert
temp_air	Alert	Fan on until back under the maximum
humidity	Alert	Fan on
level (water tank)	Alert, and block the pump	—

The ranges arrive as a small JSON file with a version number (values illustrative):

```
{"version": 12, "valid_from": "2026-09-26T00:05:00+03:00",  
  "F1": {"soil_moisture": [30, 45], "temp_air": [24, 32],  
        "humidity": [40, 80], "level": [20, 100], "pump_seconds": 120}}
```

The master asks the farm computer for the latest version at an interval the team agrees: daily is enough for the product, and every 5 minutes in the demo so a new plan shows up quickly. It only downloads when the version changes, and saves the ranges on the SD card so they survive a restart.

2.6 Safety built into the hardware

- **Actuator set:** every "on" command has a maximum run time, and the relay switches off by itself when it runs out. If the actuator set hears nothing from the master for a set time, it switches everything off.
- **Master:** keeps enforcing the last saved ranges and keeps logging to the SD card when the farm computer or the internet is down.
- **Hard limits:** ranges outside fixed safe limits, set at onboarding, are never accepted (§3.4).

Each layer keeps working when the one above it fails.

2.7 Build notes

- A radio packet holds 32 bytes: send packed numbers (IDs as single bytes, the value as a float), not text.
- The SD card and the radio both use SPI on the ESP32: give them separate chip-select pins, or separate SPI buses. Some SD modules misbehave on a shared bus.
- The ESP32 has no clock that survives a restart: get the time over WiFi (NTP) at startup. The final product adds a DS3231 clock module.
- From the schematic review: feed 5 V into the Nano Every's 5V pin rather than VIN, and add a pull-up on the DHT11 data line.
- Venue WiFi is unreliable: run a hotspot from the laptop or a phone for the master.

2.8 MVP and final

Area	MVP	Final
Fields	One demo field: a soil tray with the sensors on the current schematic	Many fields; hydroponic farms add EC, pH and water temperature
Sensor sets	One	One or more per field
Actuator sets	One: pump and fan on relays	Per field: valves, dosing pump, fans, shade motor

Area	MVP	Final
Radio	nRF24L01	A longer-range radio (e.g. LoRa) for large farms
Clock	NTP over WiFi	DS3231 clock module
Data	SD card CSV and SQLite on the laptop	Same, synced to a cloud server
Range updates	Every 5 minutes, for the demo	An agreed interval, e.g. daily
Power	USB power banks	Solar, per the team's power plan
Enclosures	Open on the table	Weatherproof, shaded boxes

3. The agentic part in depth

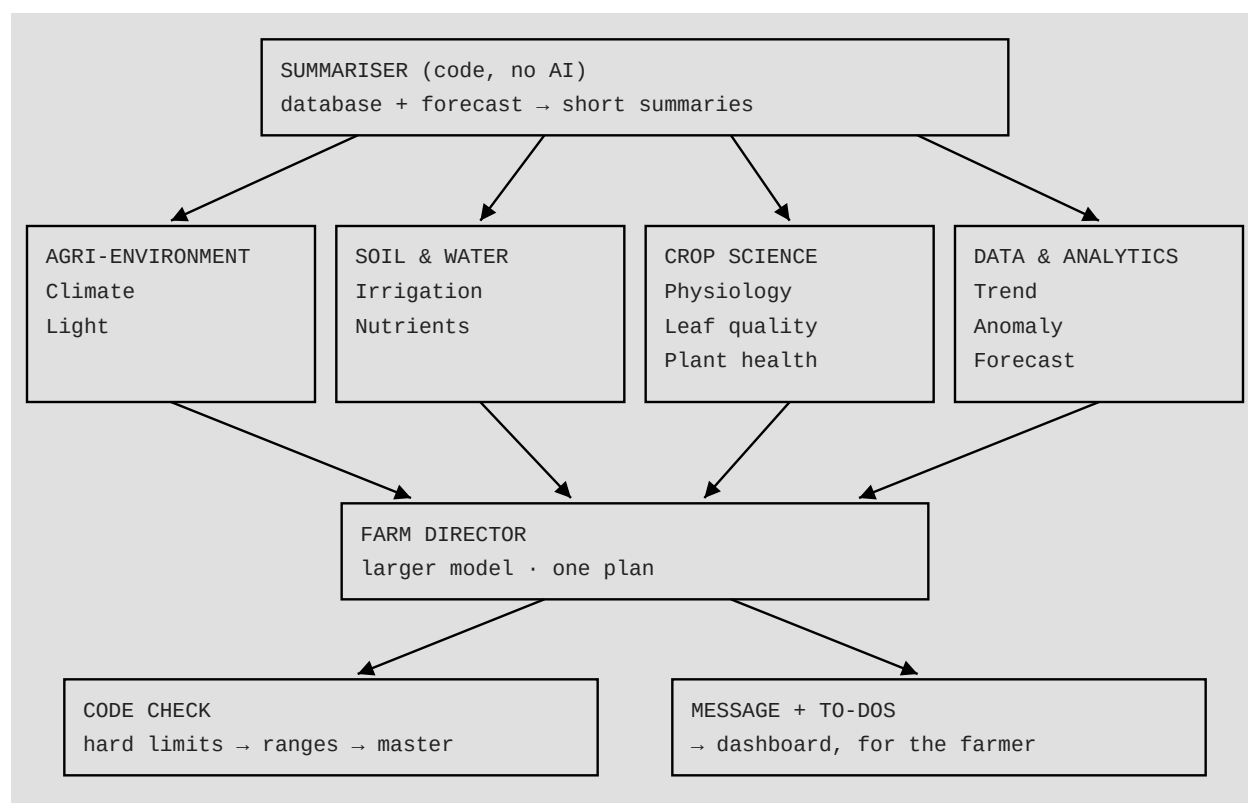


Figure 2. One run of the farm agent network. Code does the counting; agents do the judging; code checks the result before the master sees it.

3.1 Two AI stages

Stage	When	Model	Job
Onboarding assistant	Once per farm, and again when the farm changes	The most capable model we can reach	Understands the farm through conversation; designs the hardware plan, the hard limits and the agent network
Farm agent network	Every few hours	Small open models, local (Ollama)	Reads summaries of the data and the forecast; sets ranges; writes advice

3.2 The onboarding assistant

Farmers describe their farm in their own words; the assistant does the structuring. There is no form. It asks follow-up questions until it understands:

- where the farm is, and how many fields it has: size, soil or hydroponic, open air or covered;
- what grows in each field, and when it was planted;
- the water source, how water reaches the plants, and which fertilizer is used;
- where there is power and internet;
- what goes wrong most often.

Its tools: Open-Meteo for finding the location, the forecast and past weather (free for non-commercial use, no API key); a crop file with safe ranges per crop and growth stage, taken from plant science and checked by the team; and the department template in §3.3.

What it produces:

1. **Farm profile:** everything it learned, in a fixed structure.
2. **Hardware plan:** which sensor set and actuator set goes in each field, with which sensors, and where the master goes.
3. **Hard limits and starting ranges** for each crop.
4. **The agent network:** departments, roles, models and tools, written as CrewAI agent definitions.

A person on our team reviews all four before anything is installed or run.

How it builds the CrewAI network. CrewAI runs agents; it doesn't invent them. Each agent is a short definition: role, goal, backstory (its rules), model and tools, with one file per agent in a CrewAI project. The assistant fills in the department template: it keeps the departments the farm needs, drops agents the sensors can't feed, and writes each role around the farm's crops and problems. One generated definition:

```
{"role": "Irrigation agent for field F1 (purple amaranth, open-air soil bed)",  
  "goal": "Keep soil moisture in range with as little water as possible",  
  "backstory": "Hard limits 25-55 %. You get a 24-hour summary and the forecast.  
               Reply only in the Irrigation JSON format.",  
  "llm": "ollama/gemma2:2b",  
  "tools": ["field_summary", "forecast"]}
```

3.3 The farm agent network: departments and roles

This is the template the assistant adapts for each farm. It follows the team's first network, with the gaps found in the design review filled in (air quality, fertilizer, water temperature, advice for the farmer).

Department	Roles	Reads	Produces
Agri-Environment	Climate (and air quality, where there is a sensor) · Light & Photosynthesis	Temperature, humidity, light, air quality, forecast	Temperature, humidity and light ranges; heat warnings
Soil & Water	Irrigation & Salinity · Nutrients & Fertilizer	Soil moisture (or EC, pH and water temperature), tank level, pump runs, soil test, fertilizer used	Watering ranges and pump seconds; fertilizer advice
Crop Science (after the two above)	Crop Physiology · Leaf Quality & Pigment · Plant Health	The two reports above, growth stage, the farmer's notes and photos	Adjustments for the growth stage; disease and pest warnings
Data & Analytics	Trend · Anomaly · Forecast	Results from code tools	What changed, what's unusual, what's coming
Farm Director	One agent over all departments	The four department reports	One plan: ranges for the master; a message and to-dos for the farmer

Models in the team's first network: llama3.2:3b (Agri-Environment), gemma2:2b (Soil & Water), phi3.5 (Crop Science), qwen2.5:3b (Data & Analytics) and qwen2.5:7b (Farm Director). A farm that hasn't chosen a crop yet also gets a Crop Suggestion role, which proposes crops and a rotation plan.

3.4 One run, step by step

1. **Trigger:** every 6 hours, or the "Run now" button on the dashboard.
2. **Summarise (code, no AI):** a Python script reads the window from the database, the same data as the CSV, and computes minimum, maximum, average, trends and pump runs. It adds the forecast from Open-Meteo. Six hours of 10-second readings is about 2,000 rows per sensor; each department gets a few lines.
3. **Departments:** Agri-Environment and Soil & Water first, then Crop Science, then Data & Analytics. Every report follows a fixed JSON format, which Ollama enforces.
4. **Farm Director:** reads the four reports and writes the plan: ranges for each field, a short message for the farmer, and to-dos that need a person.
5. **Check (code):** every range is clamped inside the hard limits, and anything clamped is flagged on the dashboard.
6. **Save and send:** the plan goes into the database with a new version number. The master picks it up at its next check, and the dashboard shows the message and to-dos with Approve and Correct buttons.

If any step fails, nothing changes: the master keeps the last ranges.

3.5 Why a network of small models

- It runs on the farm computer: it works offline, has no per-call fee, and the farm's data stays on the farm.
- Small models are reliable on narrow jobs, so each specialist gets one job and only the data it needs.
- Every recommendation can be traced back to the agent that made it.
- Adding a sensor means adding one agent, which is what lets the assistant build a different network for each farm.
- The Farm Director is the one larger model, and it reads about ten short reports, not the raw data.

On fine-tuning: in the MVP, each specialist is shaped by its role, its data, its tools and the code checks. It is not fine-tuned. Every recommendation the farmer approves or corrects is logged, and that log becomes the fine-tuning data for each specialist in the final product.

3.6 MVP and final

Area	MVP	Final
Onboarding	Chat page in the dashboard, English, a large model; the team checks the outputs	Arabic and English, text or voice, on a phone; revisits the farm when crops or sensors change
Network design	The assistant fills in the department template	More roles to choose from, e.g. crop suggestion and pest outbreak watch
Process	CrewAI sequential process: departments run in a fixed order	Hierarchical managers, once the models are strong enough; CrewAI notes its hierarchical mode is designed for advanced models
Models	Small open models in Ollama on the laptop	Specialists fine-tuned on the farm's approval log
Schedule	"Run now" button and every 6 hours	Every few hours, plus whenever an alert fires
Farmer input	Text notes	Photos for the leaf and disease agents
Learning	Approval log	Harvest records across farms, to learn which conditions grow the best crops

4. Bringing a farm online

From the first conversation to a running farm in nine steps. The farmer talks to the onboarding assistant in steps 1, 5 and 9; our team works on site in steps 3 and 4.

Step	Who	What happens	Result
1. Talk	Farmer, with the onboarding assistant	The farmer describes the farm in their own words. The assistant looks up the location, climate and forecast, and asks follow-up questions until it understands each field.	Farm profile
2. Plan the hardware	Assistant, then our team and the farmer	The assistant proposes sensor sets and actuator sets for each field and where the master goes. Our team checks it and agrees it with the farmer.	Approved hardware plan
3. Install	Our team	Mount the sensor sets and actuator sets, wire the relays to pumps and fans, place the master near power and WiFi, and give every module its IDs.	Installed hardware
4. Test	Our team	Every sensor shows up on the dashboard, and every relay is switched once and logs activated and deactivated. What went where is written into the installation record.	Installation record
5. Build the agent network	Assistant	From the profile and the installation record, it sets hard limits and starting ranges, chooses the departments and roles the sensors can feed, and writes the CrewAI definitions.	Proposed network
6. Approve	Our team and the farmer	The network is explained in plain language, e.g. "10 specialists and a Farm Director will look after your two beds". The team checks the hard limits.	Approved network
7. Go live	The system	First agent run: ranges go to the master, which starts enforcing them.	Running farm
8. Day to day	Farmer	Checks the dashboard, gets messages, approves or corrects advice, and adds notes or photos.	Better advice over time
9. When the farm changes	Farmer, with the assistant	A new crop, field or sensor: the farmer tells the assistant, which updates the plan and the network.	Updated network

MVP: the demo runs steps 1 to 7 on a table: a short chat, one sensor set and one actuator set already wired, and a first agent run.

5. Development plan

5.1 MVP milestones

Milestone	What works at the end	Who
M1 Data pipe	A fake-sensor script posts rows to the laptop; they land in SQLite and appear on the dashboard. In parallel, the sensor set sends real packets that the master prints.	Backend; firmware
M2 Hardware loop	The master logs to the SD card, posts rows and checks fixed ranges; the actuator set switches the pump and logs activated and deactivated.	Firmware; hardware
M3 Agents make a plan	The summariser and the CrewAI network turn a day of sample data into a plan; the code check clamps it; it's saved with a version number.	AI
M4 Loop closed	The master fetches new ranges, saves them and acts on them.	Firmware; backend
M5 Onboarding	The chat produces a farm profile, a hardware plan and agent definitions, and the network runs from them.	AI
M6 Demo ready	Seeded "heatwave day" data, the worked example (§6) rehearsed, timings checked.	Everyone

M1 to M4 make a working product; M5 is the moment that impresses. Build in this order so there is always something to show.

5.2 Tools for the MVP

Part	Tools
Sensor and actuator sets	Arduino C++, RF24 radio library
Master node	ESP32 Arduino core: RF24, SD, WiFi, HTTP client, NTP
Endpoint and database	Python FastAPI (POST /log, GET /ranges), SQLite
Dashboard	Streamlit, reading SQLite
Agent network	CrewAI and Ollama with small open models; a Python summariser
Onboarding assistant	A chat page in the dashboard, a large model, Open-Meteo lookups

5.3 If time runs short

- **WiFi fails at the venue:** the master sends rows to the laptop over USB instead.
- **Onboarding isn't ready:** show its output for a fixed example farm, built from the template.
- **A full agent run is too slow on stage:** run it before the demo, then run one department live.
- **Actuator hardware breaks:** the master still logs its commands, and the dashboard shows them.

5.4 After the hackathon

Stage	Goal	Adds
Pilot	One real farm for one growing season	Weatherproof hardware, solar power, Arabic onboarding, daily range updates, approval logging
Product	Many farms	Cloud server and mobile app, longer-range radio, hydroponic sensors, photo input, fine-tuned specialists, learning from harvests

5.5 Decisions to settle

1. **Soil or hydroponic for the demo.** Recommended: a soil tray with the sensors already on the schematic; hydroponic sensors come in the final product.
2. **Intervals.** Suggested for the MVP: the master checks for new ranges every 5 minutes, and the agents run every 6 hours plus on the button.
3. **The onboarding model.** A large open-weight model through a hosted API, since onboarding happens once and quality matters most there, or the largest model the laptop can run locally.

6. Worked example: the agentic network demo

An example farm near Al Khor with two open-air soil beds of purple amaranth. The conversation, numbers and outputs are illustrative: they show how the system behaves, not results from a real farm.

6.1 The onboarding conversation (excerpt)

Who	Says
Assistant	Hello! I'll help set up the system for your farm. Where is it?
Farmer	Near Al Khor, north of Doha.
(tools)	<i>Looks up the location, the next week's forecast and past weather.</i>
Assistant	Thanks. It's still very hot there this week. What do you grow, and how?
Farmer	Purple amaranth, in two soil beds, each about 10 by 20 metres. I water with a hose from a tank.
Assistant	Are the beds in the open, or covered?
Farmer	Open air. Sometimes I put up shade cloth.
Assistant	Is there electricity and internet near the beds?
Farmer	Electricity in the shed, about 30 metres away. Mobile data works.
Assistant	What goes wrong most often?
Farmer	In the heat the leaves go pale and burn at the edges, and I never know if I'm watering too much.
Assistant	When did you plant, and do you use fertilizer?
Farmer	About three weeks ago. Granular NPK once a month.
Assistant	Here's what I suggest. Each bed gets a sensor set and a small pump on a drip line, and the master goes in the shed. The beds are in the open air, so you don't need an air quality sensor. I'll also set up 10 AI specialists and a Farm Director for your two beds. Shall I show you the details?
Farmer	Yes, go ahead.

6.2 What the assistant produced

Farm profile (excerpt):

```
{"location": "Al Khor, Qatar",  
  "fields": [  
    {"field_id": "F1", "type": "soil bed, open air", "size_m": [10, 20],  
      "crop": "purple amaranth", "planted": "2026-09-04"},  
    {"field_id": "F2", "type": "soil bed, open air", "size_m": [10, 20],  
      "crop": "purple amaranth", "planted": "2026-09-04"}],  
  "water": "tank, hose", "fertilizer": "granular NPK, monthly",  
  "power": "shed, 30 m away", "internet": "mobile data",  
  "problems": ["pale, scorched leaves in heat", "unsure how much to water"]}
```

Hardware plan:

Field	Sensor set	Sensors	Actuator set	Actuator
F1	S1	Air temperature and humidity, soil moisture, light, tank water level	A1	Drip-line pump (relay1)
F2	S2	Air temperature and humidity, soil moisture, light	A2	Drip-line pump (relay1)

The master node and the farm computer go in the shed. Only S1 measures the tank, since both beds share it.

Agent network for this farm:

Department	Roles created	Left out, and why
Agri-Environment	Climate · Light & Photosynthesis	Air quality: open-air beds, no sensor
Soil & Water	Irrigation · Nutrients & Fertilizer	Salinity: no EC sensor, so fertilizer advice only
Crop Science	Crop Physiology · Leaf Quality & Pigment · Plant Health	—
Data & Analytics	Trend · Anomaly · Forecast	—
Farm Director	One	Crop Suggestion: the crop is already chosen

Hard limits and starting values:

Setting	Hard limit (fixed)	Starting value
Soil moisture range	25–55 %	30–45 %
Pump run per command	At most 300 s	120 s
Air temperature	Alert above 32 °C	24–32 °C
Tank water level	Block the pump below 15 %	Alert below 20 %

6.3 One run: 06:00 on September 26

Input, the summary the code builds for F1:

```
{"field": "F1", "window": "last 24 h",  
  "soil_moisture": {"now": 31.0, "min": 30.2, "trend_per_h": -0.6},  
  "yesterday": {"temp_air_max": 38.6, "soil_drying_after_10am_per_h": -2.5,  
                "pump_runs": 3},  
  "forecast_today": {"temp_max": 41, "hottest_hour": "13:00"},  
  "farmer_note": "leaves on the west edge look pale"}
```

Department reports (shortened):

Department	Report
Agri-Environment	Heat stress likely from 11:00: the forecast is 41 °C against a 32 °C limit. Shade cloth from 11:00 to 15:00 would help most.
Soil & Water	The soil dries 2.5 % an hour after 10:00, and the pump ran 3 times yesterday. Raise the range to 35–50 % today and water before 08:00. No fertilizer change until the leaves are checked.
Crop Science	Week 3, leafy growth. Pale leaves on the sunny west edge point to heat and light stress more than a nitrogen shortage. Ask for a photo.
Data & Analytics	F1 has dried 40 % faster than F2 since Tuesday. Check the drip emitters in F1.

The Farm Director's ranges, before the check:

```
{"version": 13, "valid_from": "2026-09-26T06:05:00+03:00",  
  "F1": {"soil_moisture": [35, 50], "pump_seconds": 400, "temp_air": [24, 32]},  
  "F2": {"soil_moisture": [33, 48], "pump_seconds": 120, "temp_air": [24, 32]}}
```

The code check: 400 s for F1 is above the 300 s hard limit, so it is clamped to 300 s and flagged on the dashboard. Everything else is inside the limits, so version 13 is saved.

The message to the farmer (sent in Arabic and English):

"A very hot day today, up to 41 °C around 1 pm. The system will water bed 1 a little more this morning. Please put the shade cloth over both beds from 11 am to 3 pm, and check the drip line in bed 1: it's drying much faster than bed 2. Could you send a photo of the pale leaves?"

To-dos on the dashboard:

1. Shade cloth on both beds, 11:00–15:00
2. Check the drip emitters in bed 1
3. Send a photo of the pale leaves on the west edge

What the master did next:

```
F1,S1,soil,reading,soil_moisture,30.9,%,2026-09-26T06:12:10+03:00
F1,A1,relay1,action,pump,activated,,2026-09-26T06:12:11+03:00
F1,A1,relay1,action,pump,deactivated,,2026-09-26T06:17:11+03:00
F1,S1,soil,reading,soil_moisture,37.9,%,2026-09-26T06:17:20+03:00
```

At 06:12 the soil moisture was below the new minimum of 35 %, so the master ran the pump for the clamped 300 s. The actuator set logged both switches, and the soil was back inside the range five minutes later.

6.4 The demo on stage (about 5 minutes)

1. **The problem** (30 s): heat and guesswork on small farms in Qatar.
2. **Onboarding** (90 s): play the farmer and chat with the assistant; it produces the hardware plan and the agent network.
3. **Live hardware** (60 s): the dashboard shows live readings. Pull the soil probe out of the tray, the master switches the pump, and the activated and deactivated rows appear.
4. **Agent run** (90 s): press "Run now" on the seeded heatwave data. Show the department reports, the Director's plan, the clamped value and the farmer's message.
5. **Loop closed** (30 s): the master picks up version 13, and the dashboard shows the new ranges in force.

Sources

- The team's design notes (Draft 2, Sep 25), meeting recordings and whiteboard photos, the edge-device schematic, the team's agent-network table, and the architecture agreed on Sep 25.
- CrewAI: [hierarchical process](#) and [GitHub repository](#).
- Ollama: [structured outputs](#).
- [Open-Meteo](#): free weather and geocoding APIs for non-commercial use.